UNIVERSIDAD NACIONAL AUTÓNOMA DE MEXICO

Introducción a ciencias de la computación

Nicolás Bustos

Número de cuenta 427340976

Práctica de laboratorio: Entrada de datos y cierre de la cabaña del misterio

Actividad 1.1 – Dardos a la medida

¿Qué hace cada uno de estos métodos y qué valor regresa?

¿Qué pasa si se usa nextInt() y el usuario escribe algo que no es un número

entero?

¿Cómo puede ayudarte hasNextInt() a evitar ese problema?

##Visto en laboratorio

1.3 Estrategia para cambiar el programa, pedir el número al usuario y detectar errores.

Lo que hice para solucionarlo fue usar la clase Scanner, con algunas de sus utilidades para evaluar el input antes de asignarlo a una variable

Inicié un bucle while, y con un if dentro, usando hasNextBigInteger validé que el imput fuera un número entero. Luego, para asegurarme que el número cupiera en los enteros, usé hasNextInt dentro de un if anidado en el anterior. Si se cumple la condición, la guarda en la variable, y si no, explica el error al usuario y le pide otro número. Por último, con otro if más, validamos que el número sea mayor a cero. Si se cumple, termina el ciclo while, y si no, explica el error al usuario y vuelve a pedir el número.

Actividad 2.1 – Programa final para la cabaña del misterio

2.1.4 Estrategia para llevar la cuenta de las operaciones y los errores, y para terminar el programa en cada uno de los tres casos.

Para llevar la cuenta, lo que hice fue declarar dos variables enteras inicializadas en cero al principio. Inicié un bucle principal do-while, y adentro coloqué un bucle while para el menú. Con un if dentro del menú, validé que lleváramos menos de 3 errores y menos de 5 operaciones. Usando hasNextInt validé que el input fuera un entero y con otro if validamos que estuviera en el rango de 0 a 4. Si se cumple la condición, le sumo uno a la variable de operaciones y rompe el ciclo del menú; si no, explica el error al usuario, le suma uno a la variable de errores y vuelve a pedir la opción.

Para terminar el programa en cada uno de los tres casos, lo que hice fue lo siguiente:

Primero, para el caso del límite de errores, usé un else if anidado en el validador principal. Si el contador de errores es igual a 3, imprime el resumen del cliente, cambia una variable booleana (salirPorLimites) a verdadero y rompe el ciclo con un break.

Segundo, para el caso del límite de operaciones, usé un else al final que hace lo mismo: si ya no se cumplió la primera condición, imprime el resumen, activa la variable booleana y rompe el ciclo while.

Por último, para salir del programa, con otro if afuera del menú validamos si se activó la variable booleana de los límites o si la opción que ingresó el usuario es igual a 0. Si se cumple la condición, rompe el ciclo principal con un break y finaliza el programa, y si no, ejecuta la operación y al final pregunta si quiere hacer otra para volver a empezar.

2.1.5 : Los datos sí coinciden.

Actividad 2.2 – Conclusión y retrospectiva

2.2.1 Conclusión

Al principio me gustó, pero al no poder usar POO u otras herramientas que no hemos visto, se hizo **demasiado** tediosa. Fue una práctica muy larga y constantemente tuve varios errores. Algo que aprendí fue a recompilar el código más seguido para ir corrigiendo los errores a medida que salen, en vez de escribir mucho código de una sola vez, lo que provoca que puedan salir muchos más errores al mismo tiempo al compilar y se vuelve más difícil identificarlos y corregirlos.

2.2.2 A lo largo de estas prácticas...

Me he dado cuenta de lo mucho que puede facilitar las cosas usar un paradigma de programación orientado a objetos, sobre todo para este último ejercicio del cotizador del señor Pines.

2.2.3 ¿Qué tan fácil fue encontrar y corregir errores? ¿Qué partes de tu código se repiten o se parecen mucho entre sí?

En casi todas las ocasiones fue fácil encontrar los errores porque el compilador decía la línea en la que se encontraba; La única ocasión en la que fue muy difícil fue cuando tenía un error en un printf, en el que pedía imprimir una variable entera con un %.2f reservado para doubles, ya que el compilador no me mencionaba dónde estaba el error.

Las partes de mi código que se repetían mucho entre sí o se parecían, eran las impresiones en pantalla que debía hacer, y el control de errores en cada dato escrito por el usuario.